Avionics for Rocket — Conversation Export

Raspberry Pi GPS, sensors, LoRa, and camera setup.
Exported 8 October 2026.

This document is a readable transcript of the working session: what failed, what was changed, and the commands and scripts that ended up working. Short acknowledgements are omitted.

---

1. GPS NMEA parser — UTF-8 crash

The first script read `/dev/ttyS0` at 9600 and decoded each line as UTF-8. `readline()` returns raw bytes. GPS noise and partial lines are not valid UTF-8, so `decode('utf-8')` raised `UnicodeDecodeError`.

Working pattern:

```
line = raw.decode("ascii", errors="ignore").strip()
if not line.startswith("$"):
    continue
```

Catch `pynmea2.ParseError` so one bad sentence does not kill the loop. `msg.num_sats` on a GGA sentence is satellites used in the fix. `msg.gps_qual` of 0 means no fix.

NMEA latitude and longitude are degrees and decimal minutes, not decimal degrees. `2819.24877,N` is 28 degrees and 19.24877 minutes north:

```
28 + (19.24877 / 60) = 28.320813 N
80 + (44.49880 / 60) = 80.741647 W
```

West is negative, so the map form is `28.320813, -80.741647`. `pynmea2` already provides `msg.latitude` and `msg.longitude`. `msg.lat` and `msg.lon` are the raw fields.

---

2. Binary garbage and the Pi 3 UART

`print(raw)` showed bytes like `b'\xc0\xe3\x9c\x88SEE...'`, not `$GPGGA`. That is a baud or UART-clock mismatch.

On a Pi 3, GPIO 8/10 are the mini-UART (`/dev/ttyS0`) unless Bluetooth is disabled. Its baud follows the core clock. `/dev/ttyAMA0` is the stable PL011 UART.

`/boot/config.txt` under `[all]` needs:

```
enable_uart=1
dtoverlay=disable-bt
```

Remove `console=serial0,115200` from `/boot/firmware/cmdline.txt`. Keep `console=tty1`. After reboot, `/dev/serial0` should point at `ttyAMA0`. Bluetooth is disabled by that overlay.

The file that says `DO NOT EDIT THIS FILE` is a stub. The real files are `/boot/firmware/config.txt` and `/boot/firmware/cmdline.txt`. Edit them with `sudo nano`.

A baud sweep at 4800, 9600, 38400, 57600, and 115200 is the way to find NMEA. A good line starts with `b'$GP` or `b'$GN`.

---

3. Serial login prompt stealing the port

Even with `enable_uart=1`, `serial-getty@ttyAMA0.service` was active. The log showed failed logins for `$GNGGA`, so the GPS was already sending a fix and the login prompt was eating it. Getty also kept resetting `/dev/ttyAMA0` to `crw------- root root`, which caused `Permission denied` from both Thonny and the venv Python.

`systemctl disable` was not enough. The unit came back as `enabled-runtime` because `/boot/firmware/cmdline.txt` still contained `console=serial0,115200`.

Fix:

```
sudo systemctl stop serial-getty@ttyAMA0.service
sudo systemctl disable serial-getty@ttyAMA0.service
```

Remove `console=serial0,115200` from `/boot/firmware/cmdline.txt`, then reboot.

After reboot:

- `/dev/serial0 -> ttyAMA0`
- `systemctl is-active serial-getty@ttyAMA0.service -> inactive`
- `/dev/ttyAMA0` is `crw-rw---- root dialout`
- User `pi` is in group `dialout`, so no `chmod` is required

A live read then returned:

```
$GNGGA,145743.000,2819.24962,N,08044.49755,W,1,16,0.7,6.2,M,-29.7,M,,*7E
$GPTXT,01,01,01,ANTENNA OK*35
```

That is 14:57:43 UTC, about 28.3208 N, 80.7416 W, GPS fix, 16 satellites, altitude 6.2 m. `$GNZDA` is time and date only, not position.

Debug print that shows every line, including non-GGA:

```
import serial
serialPort = serial.Serial("/dev/serial0", 9600, timeout=1)
while True:
    print(serialPort.readline())
```

---

4. Python packages and the virtual environment

Current Raspberry Pi OS blocks system-wide `pip` (PEP 668). `pynmea2` is not an apt package. `python3-serial` is.

Environment used for this project: `~/libs-venv`. The name is only a folder name. `gps-venv` was created earlier and left unused. Do not rename a venv with `mv`. Scripts in `bin/` contain the old path.

```
python3 -m venv ~/libs-venv
source ~/libs-venv/bin/activate
pip install pynmea2 pyserial
```

`pip` inside the activated environment is the right installer. `pip3` from a normal shell hits system Python and is blocked. The import name for the serial package is `serial`. The pip name is `pyserial`.

Thonny must use this interpreter:

```
/home/pi/libs-venv/bin/python3
```

Tools, Options, Interpreter, Alternative Python 3 interpreter. `python3` and `python3.13` in that folder are the same interpreter. Quit Thonny completely after changing it.

`pyLoraRFM9x` is only needed for the RFM9x radio. It failed at first because `lgpio` needs `swig` to build:

```
sudo apt install swig python3-dev liblgpio-dev
pip install pyLoraRFM9x
```

Optional old behavior, not used here: `pip3 install --break-system-packages ...`

---

5. Copying the project folder

VNC file transfer is not available on the Pi's bundled server. Copy with `scp` from the PC.

PC to Pi:

```
scp -r "$env:USERPROFILE\Desktop\Avionics for Rocket"
pi@raspberrypi.local:/home/pi/Desktop/
```

Pi to PC:

```
scp -r "pi@raspberrypi.local:/home/pi/Desktop/Avionics for Rocket" "$env:USERPROFILE\
Desktop\"
```

Quotes are required because the folder name has spaces. SSH must be enabled on the Pi.

---

6. MPU-6050

I2C was already enabled (`dtparam=i2c_arm=on`). User `pi` is in group `i2c`. SCL is pin 5 (GPIO 3). SDA is pin 3 (GPIO 2).

```
sudo apt install i2c-tools
i2cdetect -y 1
```

The chip answered at `0x68`. Install `smbus2` in the venv. Import with `import smbus2 as smbus`, not `from smbus2 import smbus`.

Who-am-I register `0x75` returned `0x68`. Power register `0x6B` returned `0x0` after wake. A first temperature read of exactly `36.53` means the raw register was 0 (formula zero point). The next read was raw `-3552`, about `26.1 C`. Flat and still, Z acceleration should be near `9.8` m/s^2 and gyro values near 0.

---

7. HMC5883L

Same I2C pins. Scan showed `1e`, which is a real HMC5883L, not a QMC5883L clone (`0d`). Data register order is X, then Z, then Y. A sample read returned `379 -311 34`, then a later sample `229 145 -101`.

Heading from `atan2(y, x)` assumes the board is level. The ElectronicWings declination of `-0.00669` radians is about `-0.4` degrees, not Cocoa. Local declination is about `-6.5` degrees, or `-0.113` radians. Use `0` for a bench test.

The MPU-6050 (`0x68`), HMC5883L (`0x1e`), and BMP280 (`0x76`) can share SDA and SCL. Addresses do not collide. The GPS stays on the UART, pins 8 and 10. Three breakout pull-up sets are usually fine. Keep the magnetometer away from power wires.

---

8. BMP280

Scan showed `0x76`. Chip id register `0xD0` returned `0x58`, so this is a BMP280 (temperature and pressure, no humidity). A BME280 would return `0x60`. Power it from 3.3 V.

Package name is `bmp280`, not `bmp280-python`:

```
source ~/libs-venv/bin/activate
pip install bmp280
```

```
from smbus2 import SMBus
from bmp280 import BMP280
from time import sleep

chip = BMP280(i2c_dev=SMBus(1), i2c_addr=0x76)
sleep(1)
print(chip.get_temperature(), chip.get_pressure(), chip.get_altitude())
```

The first sample was `23.6 C`, `724` hPa, `3009` m. That pressure was an incomplete conversion. Later samples were about `24.8 C` and `1011.7` hPa, altitude about `13` m against the default sea-level pressure of `1013.25` hPa. That fits Cocoa. Zero altitude on the pad before launch. Weather moves sea-level pressure between about `1010` and `1020` hPa.

---

9. LoRa / pyLoraRFM9x

SPI devices were present: `/dev/spidev0.0` and `/dev/spidev0.1`. The board labels were an Adafruit-style breakout: VIN, GND, EN, G0, SCK, MISO, MOSI, CS, RST.

Physical pin numbers and BCM numbers are different labels for the same holes. Use SPI0, not the SPI1 block at the right edge of the pinout.

RFM pin	Physical pin	BCM
MOSI	19	10
MISO	21	9
SCK	23	11
CS	26	7 (CE1)
RST	22	25
G0	29	5
VIN	1	3.3 V
GND	6	ground

`EN` is pulled high on the Adafruit board, so it can stay unconnected. Antenna must be on the ANT pad before transmit. A 915 MHz whip or a 78 mm wire is enough for a bench test. US frequency is 915 MHz.

First init failed with `AssertionError: LoRa initialization failed` because SPI did not read back sleep mode. That is chip-select or MOSI/MISO/SCK, not frequency. After the CS wiring was corrected, `lora.send("hello", 255)` printed `sent`, and a second board received it.

```
from pyLoraRFM9x import LoRa, ModemConfig

lora = LoRa(
    spi_channel=1,
    interrupt_pin=5,
    reset_pin=25,
    spi_port=0,
    my_address=1,
    freq=915,
    tx_power=14,
    modem_config=ModemConfig.Bw125Cr45Sf128,
    acks=False,
)
lora.send("hello", 255)
print("sent")
```

`sent` only means this Pi handed the bytes to the radio. The other board must use the same modem config.

---

10. Camera

`rpicam-hello --list-cameras` showed an ov5647, the Camera Module v1, at 2592x1944. The old `picamera` module does not work. The current library is `picamera2`.

```
sudo apt install python3-picamera2
rpicam-still -o ~/Desktop/cam_test.jpg
```

Do not activate the venv for the apt install. `apt` ignores it. A 154 KB JPEG confirmed the capture path.

`picamera2` is a system package. So the venv can see it and the pip libraries at the same time, this was set:

```
sed -i 's/include-system-site-packages = false/include-system-site-packages = true/'
~/libs-venv/pyvenv.cfg
```

Thonny stays on `/home/pi/libs-venv/bin/python3`. Check:

```
~/libs-venv/bin/python -c 'from picamera2 import Picamera2; print("ok")'
```

Five stills, two seconds apart, in a timestamped Desktop folder. A slash cannot be in a folder name.

```
from datetime import datetime
from pathlib import Path
from time import sleep
from picamera2 import Picamera2

stamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
folder = Path("/home/pi/Desktop") / f"photos_{stamp}"
folder.mkdir(parents=True, exist_ok=True)

cam = Picamera2()
cam.configure(cam.create_still_configuration())
cam.start()
sleep(1)

for i in range(1, 6):
    path = folder / f"cam_{i}.jpg"
    cam.capture_file(str(path))
    print("saved", path)
    if i < 5:
        sleep(2)

cam.stop()
print("done", folder)
```

A successful run saved `cam_1.jpg` through `cam_5.jpg` in `/home/pi/Desktop/photos_2026-10-07_20-04-38/`. The `INFO` lines from libcamera are normal startup, not errors.

---

11. Working addresses and ports

Device	Bus	Address or port
GPS	UART /dev/serial0 -> ttyAMA0	9600 baud
MPU-6050	I2C bus 1	0x68
HMC5883L	I2C bus 1	0x1e
BMP280	I2C bus 1	0x76, chip id 0x58
RFM9x LoRa	SPI0 CE1	915 MHz, G0 GPIO 5, RST GPIO 25
Camera	CSI	ov5647

Python for the combined script: `/home/pi/libs-venv/bin/python3`.